Supplementary Material for

Biomedical Knowledge Graphs and Large Language Models:
A Critical Engineering Survey

Rangan Das Sounak Ghosh Sushmit Dasgupta

Utsav Bandyopadhyay Maulik Ujjwal Maulik Sanghamitra Bandyopadhyay

Abstract

This document covers the reporting package for a graph-grounded language system, a scale for grading reproducibility, a data card for a biomedical graph, the criteria for each step up the evidence-maturity ladder, a privacy architecture, the signals worth monitoring after deployment, and the limits of the standard benchmarks.

1 What has to be pinned down

Behavior here depends on a knowledge substrate that changes independently of the code. Construction scripts for widely used biomedical graphs get revised after publication, terminologies issue scheduled releases, and hosted model endpoints are replaced without notice. Two groups can run the same published pipeline against the same named graph and get different answers, because one of them downloaded it six months later.

So the artifact to specify is the configuration, not the model: $C = (M, P, G, T, R, V, I)$, being the model and revision, the prompt and tool policy, the graph snapshot and schema, the terminology releases, the retrieval policy, the verification layer, and the interface. Changing any element produces a new system unless equivalence is shown.

Errors also do not stay local, which is what Fig. S1 shows. A wrong edge admitted at construction is indexed like any other, retrieved when relevant, and cited accurately by a generator with no way to know it is wrong.

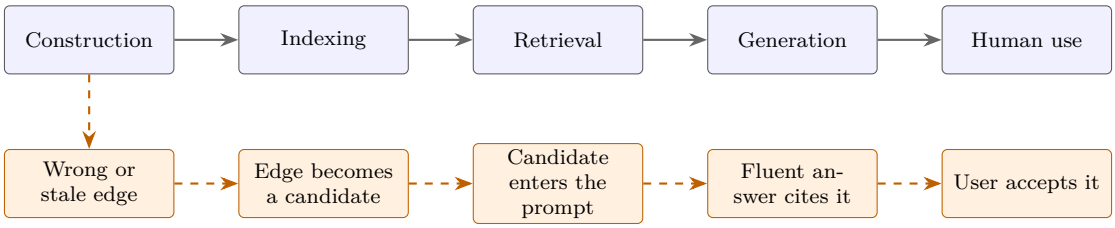

Figure S1: **Why a single bad edge is hard to catch downstream.** The upper row is the intended pipeline; the lower row follows one incorrect edge through it. Every step behaves correctly given its input, so nothing raises an alarm. Reporting therefore has to cover provenance and snapshot identity, not only model performance.

2 The reporting package

What another group needs in order to reconstruct both the knowledge boundary and the retrieval behavior of a published system. Table S1 gives the items by category. Three deserve emphasis. The graph is the part most often under-specified, and a schema that cannot express negation or population drops them silently. Per-instance retrieval traces are the only way to tell a system that retrieved the right evidence and misused it from one that never retrieved it. And if any evaluation question was generated from the graph, that has to be stated, because agreement with the graph that supplied the question is close to guaranteed.

Releasing a final answer and an aggregate metric makes retrieval error and circular evidence impossible to audit, so the log needs a record per evaluation instance. Table S2 gives the fields. Where licensing

Table S1: **The reporting package, by category.** Each row lists what has to be released for that part of the system to be reconstructable. The knowledge and retrieval rows are where published work most often falls short, and they are also the rows that determine whether a reported result can be checked for leakage after the fact.

Category	Report
Knowledge	Graph snapshot or a deterministic rebuild workflow, with checksums; schema including relation directionality and permitted qualifiers; every ontology and terminology release with edition and extension; source list with per-source versions and integration rules; licenses and redistribution constraints; graph statistics by node and relation type
Cohort and benchmark	Inclusion and exclusion criteria with the code that builds the cohort; index-time definition and split logic; exact benchmark release; lineage of any question generated from the graph, reported separately; contamination assessment against the model's training cutoff
Retrieval	Entity linker and version; embedding model and index version; query templates and every generated formal query; retrieval depth, reranking, context budget; retrieved node, edge, and path identifiers with scores per test instance; failure and timeout logs
Model	Exact identifier and revision, provider, access date; prompt templates and system messages; decoding parameters and context window; fine-tuning data and checkpoints where releasable
Evaluation	Raw outputs; atomic-claim decomposition and claim-to-evidence labels; automated judge model, version, prompts, and agreement with human labels on a calibration sample; clinician rubric, adjudication protocol, inter-rater agreement; analysis scripts, intervals, and all planned and exploratory outcomes
Operations	Environment with lockfile or container; run time, token use, cost assumptions; seeds and number of repeated runs
Change record	Graph updates, retractions, endpoint changes, prompt amendments, and benchmark corrections made after the experiment

or patient privacy prevents release, provide hashes, synthetic examples, access instructions, and a data dictionary. Licensed terminology strings should not be redistributed beyond their terms, and credentialed clinical data should never appear in a public trace.

Table S2: **Fields for a per-instance evaluation record.** Each row is one field of a machine-readable log entry written once per evaluation question. The purpose is to make every reported score traceable to the evidence that produced it, so that a reviewer can distinguish a retrieval failure from a reasoning failure without rerunning the system. The temporal and snapshot fields are what allow a result to be checked for leakage after publication.

Field	Purpose
`question_id`, `benchmark_version`	Identifies the item and the exact release it came from
`patient_or_source_group`	Supports clustered analysis and prevents split contamination
`index_time`	Fixes the as-of boundary for any temporal claim
`graph_snapshot`	Ties the answer to a specific knowledge state
`linked_entities`	Exposes entity-linking error, the most common upstream failure
`executed_queries`	Allows a reviewer to check semantic fidelity to the question
`retrieved_nodes_edges_paths`	Separates retrieval failure from reasoning failure
`source_documents_and_dates`	Enables citation entailment checking
`model_and_prompt_version`	Identifies the generator configuration
`raw_response`	Permits re-scoring under a different rubric
`atomic_claims`, `claim_evidence_labels`	Support claim-level rather than answer-level accuracy
`final_score`	The reported outcome for this item
`latency_tokens_cost`	Allows benefit to be normalized by resource use
`error_category`	Makes failure analysis aggregable across instances

3 Grading reproducibility

Reported as a binary, reproducibility loses information. A paper that describes an architecture and releases nothing differs from one that releases code but omits the graph snapshot, and both differ from one whose estimates an independent group has reproduced. Table S3 grades these separately.

The top level is what matters for clinical claims. Reproducing a number on the same data shows the pipeline was described accurately. Reproducing the direction and magnitude on a later time window, another institution, or a newer snapshot shows the finding survives what actually changes in deployment. Most current work reaches R1 or R2. Proprietary model use does not preclude a good grade, but it requires dated identifiers, preserved raw outputs, repeated runs, and a plan for endpoint retirement.

Table S3: **A five-level scale for reporting how reproducible a study is.** The levels are cumulative and describe evidence rather than intent, so a paper qualifies for a level only if someone outside the author group could perform the corresponding action. R0 to R2 concern whether the work can be rerun; R3 and R4 concern whether it survives being rerun. The gap between R2 and R3 is where most published work currently stops, and the gap between R3 and R4 is what separates a reproducible experiment from a durable finding.

Level	Criterion
R0 described	Architecture and results are described; data and code are unavailable.
R1 partially inspectable	Prompts or code are available, but the graph snapshot, retrieval traces, or model revision is missing.
R2 re-executable	Code, environment, accessible data, and a versioned graph together allow the published pipeline to be rerun.
R3 result reproducible	An independent rerun reproduces the main estimates within predeclared tolerances.
R4 transport reproduced	The direction and clinically material magnitude persist on a later time window, another institution, or a newer snapshot.

4 A data card for a biomedical knowledge graph

Naming a graph is not describing it. Two studies reporting the same well-known resource can differ in snapshot date, terminology release, the proportion of edges that were curated rather than text-mined, and the fraction of mentions that failed to resolve. Any of these changes the answer to a clinical question, and none is visible from the resource name. Table S4 asks for the properties that determine what the graph can answer. Provenance, temporal coverage, and coverage over the evaluated entities matter more than size, since four million relations are useless for a question whose decisive entity is absent.

Table S4: **Data card fields for a biomedical knowledge graph used in a language-model system.** The fields are grouped by what they let a reader determine: whether they hold the same object as the authors, what the graph's edges actually mean, whether an assertion can be traced and dated, whether the graph covers the evaluated question, and what may lawfully be redistributed. A study that omits the identity and provenance groups cannot be compared with another study on the same resource.

Group	Fields
Identity	Graph name, release, retrieval date, checksum, persistent identifier
Semantics	Node and relation schema, including directionality and permitted qualifiers
Composition	Upstream sources, source-specific versions, integration rules
Edge origin	Whether each relation is curated, rule-based, text-mined, inferred, or model-generated
Evidence	Claim-level provenance and how evidence strength is represented
Time	`valid_from`, `valid_to`, publication date, snapshot date
Coverage	Coverage over the evaluated entities, relations, and gold answers
Normalization	Ontology and terminology releases, mappings, unresolved-mention rate
Access	License, redistribution constraints, access procedure, permitted territories
Known bias	Demographic, geographic, linguistic, and publication skew
Maintenance	Update policy, retraction handling, supersession semantics

5 What each step up the maturity ladder demands

Each gate tests a property the level below left unexamined, which is why a system can be excellent at one level and fail at the next.

Gate A, analytical validity (E1 to E2). Freeze the whole configuration before testing. Compare against closed-book, long-context, vector-retrieval, symbolic, and oracle-evidence arms, and evaluate the graph, retrieval, reasoning, and generation stages separately. Use temporal splits. Report intervals and clinically weighted error severity rather than accuracy alone, since a system that is slightly less accurate but never misses a contraindication may be the better one.

Gate B, transportability (E2 to E3). Validate on an independent institution or a genuinely later period. Audit terminology mappings, missing-data mechanisms, graph coverage, guideline version, subgroup performance, and calibration. A fresh sample from the same pipeline is much weaker evidence than a changed population, because the pipeline is often what the model learned.

Gate C, operational readiness (E3 to E4). In silent deployment, demonstrate availability, tail latency, timeout behavior, data freshness, safe degradation, audit completeness, and incident detection. Complete privacy, security, safety, usability, and regulatory assessment before any output reaches a user, since that is when a technical failure becomes a clinical one.

Gate D, safe human use (E4 to E5). Predefine who may see, accept, reject, edit, or act on an output. Measure reliance, verification time, alert burden, work redistribution, escalation, and the error types users fail to catch, which are not the errors the system makes most often. Measure the work the system creates, not only the benefit.

Gate E, sustained benefit (E5 to E6). Establish comparative value, version-specific release criteria, monitoring thresholds, rollback drills, incident review, and periodic independent audit. Revalidate after any material configuration change. This gate never closes, which is why E6 is an operating state rather than an achievement.

6 Designing for privacy when the graph is the disclosure risk

A patient graph can be more disclosive than the record it came from. Linking a rare diagnosis to a date, a location, a family relation, and a genomic variant can identify an individual after every direct identifier is gone, because the combination narrows the population even when no single field does. The attack surface extends past the graph to embeddings, vector indexes, cached prompts, retrieved subgraphs, logs, and human feedback, all of which belong in the data inventory.

Legal obligations vary by jurisdiction, and meeting them is necessary rather than sufficient: a compliance label does not stop a graph from leaking structure. Fig. S2 gives the architecture. Separate the population-level graph from patient graphs, authorize at query time against the smallest subgraph that answers the question, and treat anything derived from a patient graph as inheriting its sensitivity.

The remaining controls are unglamorous. Encrypt in transit and at rest with key separation. Suppress protected data from prompts, telemetry, and vendor retention unless explicitly authorized, and document the data flow for every processor and model provider. Set retention limits for prompts, retrieved context, embeddings, logs, and feedback. Filter output and control egress against cross-patient disclosure. Provide auditable consent, access, correction, and deletion workflows where they apply. Then attack the result with membership, reconstruction, and linkage tests. Federated deployment reduces central data movement without guaranteeing privacy, since model updates, gradients, graph statistics, and query patterns can each leak, so evaluate any privacy-enhancing technology against a stated threat model rather than adopting it as a label.

7 Monitoring a system whose knowledge keeps changing

A single accuracy measure is too slow and too coarse: too slow because it aggregates over a window, too coarse because slack at one stage masks a deficit at another. A degradation that starts in the graph will not show at the output for some time. Monitoring therefore has to be stage-local, and Table S5 gives the signals.

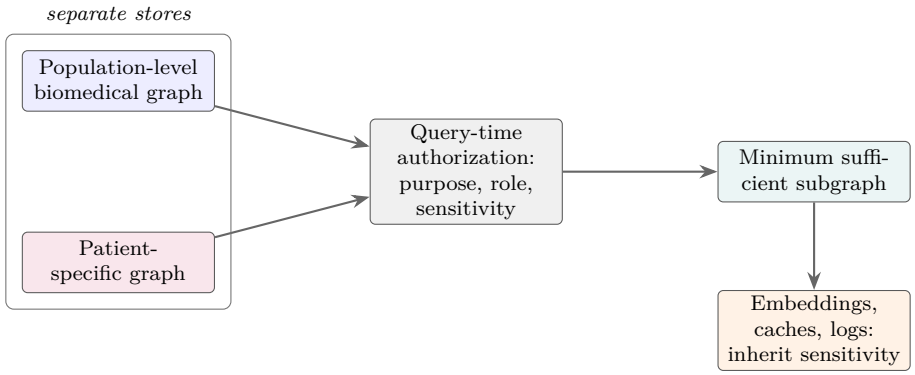

Figure S2: **Minimum privacy architecture for a patient-facing graph system.** The population-level graph and patient graphs are kept in separate stores, so that a query cannot traverse from one patient into another through a shared index. Authorization happens at query time against purpose, role, and data sensitivity, rather than being applied as a filter after retrieval, because a filter downstream of a retrieval that already crossed a boundary has nothing left to protect. The orange node records the requirement that is most often overlooked: embeddings, caches, and logs derived from a patient graph carry the same sensitivity as the graph, and therefore the same retention limits and egress controls.

What matters more than the list is what happens when a signal moves. Set every threshold in advance and map it to a response: investigate, roll back, suspend, or fall back to a restricted mode. Rerun sentinel cases after every material configuration change, since a series of individually harmless updates can drift a system a long way without tripping any single threshold. For a dynamic evidence graph, retrieve from approved immutable snapshots and promote a new one only after regression testing; retrieving from a live graph means behavior changes without a release.

Incident response needs traceability both ways. From a harmful output back to the inputs, mappings, retrieved edges, model, policy, and user actions. From a corrupted source forward to every release and output that depended on it. A system with only the first can explain one failure and cannot bound its blast radius.

Table S5: **Post-deployment monitoring signals, grouped by pipeline stage.** The grouping mirrors the evaluation layers of the main paper so that a monitoring alert points at a stage rather than at the system as a whole. Signals in the upper rows are leading indicators: unmapped concepts and retraction backlog move before answer quality does, which is what makes them worth instrumenting. Signals in the lower rows are lagging but decisive, since a rise in override rate or adverse events is evidence that earlier thresholds were set too loosely.

Stage	**Signals**
Input and mapping	Missingness, out-of-range values, unmapped concepts, ambiguity rate, language and site shift
Graph	Source freshness, retractions processed, provenance completeness, schema violations, edge-confidence distribution, update failures
Retrieval	No-evidence rate, sentinel-case recall, graph-versus-text source mix, contraindication retrieval, subgraph size, latency
Generation and verification	Unsupported-claim rate, citation entailment, contradiction rate, abstention rate, verifier disagreement, unsafe fallback events
Human use	Acceptance, override, and edit rates, verification time, automation-bias indicators, workarounds, complaints, near misses
Outcome and equity	Task success, adverse events, workflow time, decision changes, subgroup calibration and error severity
Operations and security	Availability, tail latency, timeouts, cost, access anomalies, injection attempts, data-leakage indicators, unauthorized changes

8 Where the standard benchmarks mislead

Each benchmark measures something narrower than its name suggests, so no single one validates a graph-grounded clinical system. Extraction benchmarks do not establish the accuracy of the integrated graph.

Link prediction does not establish that a novel hypothesis is true, still less that a drug works. Medical question answering does not establish workflow performance, patient outcomes, or freedom from memorization. Retrieval benchmarks do not isolate a graph-derived benefit without a matched text-retrieval arm. Graph-comprehension benchmarks say nothing about clinical validity. Domain graph reasoning agrees with a source graph, errors included. Temporal and contamination-resistant benchmarks are early evidence, not prospective safety.

Three mismatches recur. *Task mismatch* is using exam-style question answering to support a claim about clinical diagnosis, or link prediction to support one about drug efficacy; the benchmark is fine, the inference is not. *Knowledge mismatch* is generating evaluation questions from the same graph supplied at inference, which makes agreement nearly automatic and says little about correctness beyond it. *Interface mismatch* is serializing graph facts in a format that suits one model family, then reporting the result as a property of the method rather than of the pairing.

9 Leakage

Most leakage routes here inflate the graph arm rather than both arms equally, which makes leakage a threat to the attribution of the benefit and not just its size. If evaluation questions were generated from the inference graph, the graph arm is scored on reconstructing its own input while the closed-book arm is not. The measured benefit is then an artifact of the design, and no downstream statistical care recovers from it.

Table S6 lists the mechanisms and controls. Two have no analogue in conventional evaluation. *Ontology leakage* occurs when a label sits in the definitions or synonyms supplied to the model, turning classification into lookup. *Model-mediated circularity* occurs when one model family builds the edges, writes the questions, answers them, and judges the answers; correlated errors then read as agreement, and self-consistency checks amplify rather than detect it.

The concern is empirical. Models trained with medically harmful misinformation retained ordinary benchmark performance, so the benchmarks missed a clinically important degradation, and removing train-test redundancies changes embedding-model results on biomedical graphs. A random held-out edge does not simulate future discovery.

Table S6: **Leakage mechanisms, their consequences, and the controls that address them.** Each row is a distinct route by which information about the answer reaches the system before it should. The mechanisms are ordered roughly from most to least widely recognized, and the last four are specific to graph-grounded systems. Most of these routes inflate the graph arm without inflating the baseline, which is why leakage here threatens the attribution of the benefit and not merely the size of it.

Mechanism	**Consequence**	**Control**
Parametric benchmark contamination	Inflated closed-book scores; the retrieval effect becomes uninterpretable	Post-cutoff or private evaluation; report cutoff and API date; memorization probes; include a no-context arm
Hyperparameter and test reuse	Reported gains reflect adaptation to the test set	Freeze a development set; preregister the comparison; evaluate once on held-out data
Patient or encounter leakage	Models exploit patient identity or post-outcome information	Split by patient; define index time; censor features and edges; audit timestamps
Temporal leakage	Retrospective performance overstates real-time capability	Freeze all sources at the prediction date; use temporal splits; distinguish valid from transaction time
Graph-to-answer leakage	The system reconstructs its input resource rather than independent truth	Independently authored questions; disclose generation lineage; report graph-derived and external items separately
Inverse or redundant relations	Link-prediction scores measure trivial reconstruction	Remove inverse, symmetric, duplicate, and logically entailed test edges
Ontology leakage	Classification degenerates into label lookup	Redact label-bearing fields; measure lexical overlap; evaluate unseen concepts
Model-mediated circularity	Correlated model errors appear as agreement	Separate construction, generation, and judging models; use human adjudication; retain disagreement